

Assignment 5

Arpa Banik

2024-03-25

```
setwd("/Users/arpabanik/Library/CloudStorage/OneDrive-BentleyUniversity/R-Projects/MA 347/Assignment 5")
ebay <- read.csv("~/Library/CloudStorage/OneDrive-BentleyUniversity/R-Projects/MA 347/Assignment 5/eBayAuctions.csv")
```

```
# Load necessary libraries
library(rpart)
library(caret)
```

```
## Loading required package: ggplot2
```

```
## Loading required package: lattice
```

```
library(rattle)
```

```
## Loading required package: tibble
```

```
## Loading required package: bitops
```

```
## Rattle: A free graphical interface for data science with R.
## Version 5.5.1 Copyright (c) 2006-2021 Togaware Pty Ltd.
## Type 'rattle()' to shake, rattle, and roll your data.
```

```
library(rpart.plot)
library(gains)
```

a. Data preprocessing

```
# Convert Duration to a categorical variable
ebay$Duration <- as.factor(ebay$Duration)

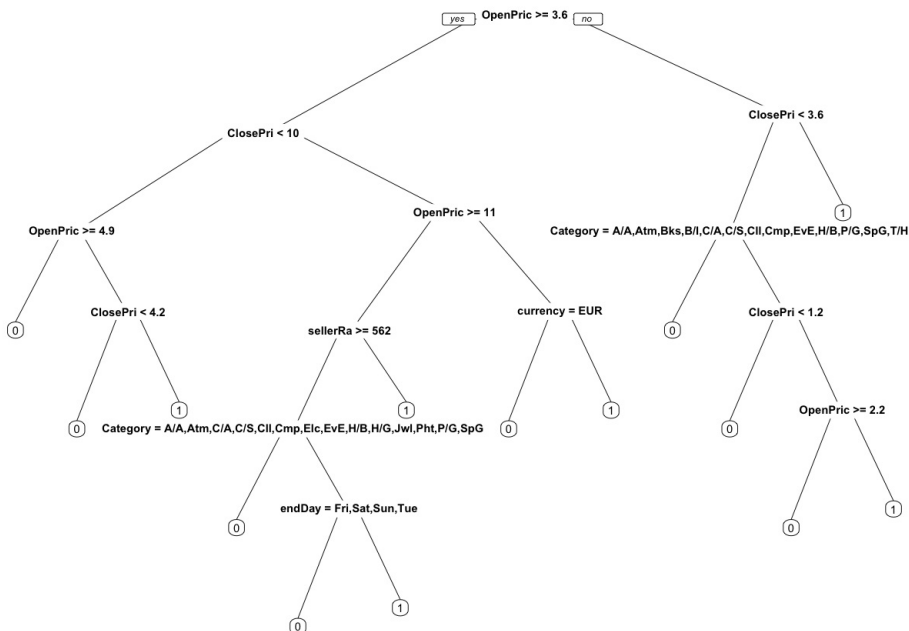
# Split the data into train (60%) and test (40%) sets
set.seed(123)
train_index <- sample(c(1:dim(ebay)[1]), dim(ebay)[1]*0.6)
train_data <- ebay[train_index, ]
test_data <- ebay[-train_index, ]
```

b. Fit a classification tree and perform 10-fold cross-validation

```
set.seed(123)
tr<- rpart(Competitive ~ ., data = train_data,method="class")
printcp(tr)
```

```
##
## Classification tree:
## rpart(formula = Competitive ~ ., data = train_data, method = "class")
##
## Variables actually used in tree construction:
## [1] Category    ClosePrice  currency    endDay      OpenPrice
## [6] sellerRating
##
## Root node error: 547/1183 = 0.46238
##
## n= 1183
##
##      CP nsplit rel error  xerror   xstd
## 1  0.297989      0  1.00000 1.00000 0.031350
## 2  0.100548      1  0.70201 0.70384 0.029461
## 3  0.063071      2  0.60146 0.60329 0.028200
## 4  0.053016      4  0.47532 0.52102 0.026889
## 5  0.031079      5  0.42230 0.48995 0.026321
## 6  0.020110      7  0.36015 0.43327 0.025167
## 7  0.017367      8  0.34004 0.40585 0.024550
## 8  0.014625     10  0.30530 0.35649 0.023330
## 9  0.013711     11  0.29068 0.31627 0.022218
## 10 0.010000     13  0.26325 0.29982 0.021728
```

```
#Plot the full tree
prp(tr)
```



c. Choose the optimal complexity parameter cp

```
lowest_cp<-tr$cptable[which.min(tr$cptable[, "xerror"]), "CP"]
lowest_cp
```

```
## [1] 0.01
```

d. Report the 10-fold CV error rate of the optimal cp

```
optimal_cp_error<-min(tr$cptable[,4])
optimal_cp_error
```

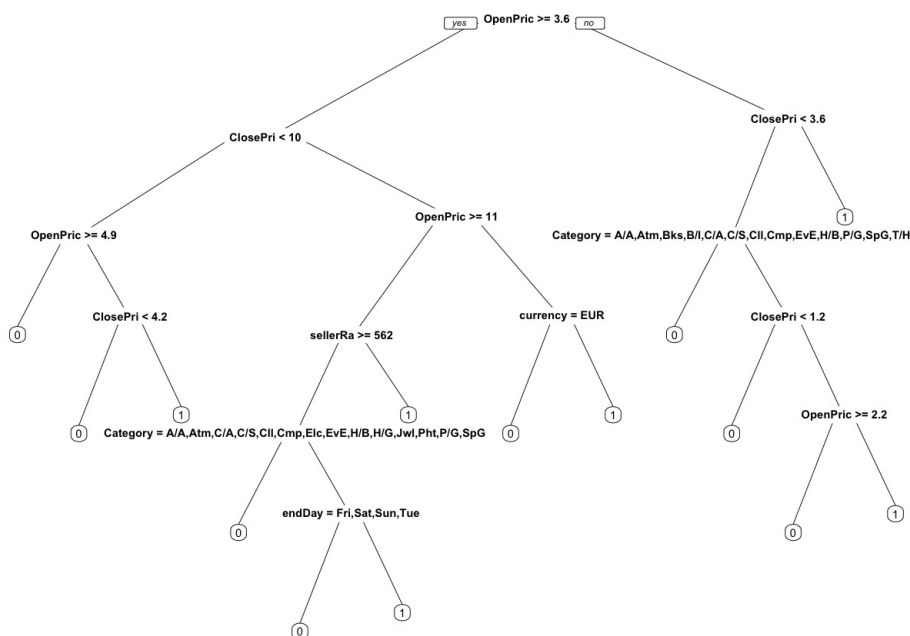
```
## [1] 0.2998172
```

e. Prune the tree according to the optimal cp

```
pruned.tr <- prune(tr, cp = lowest_cp)
printcp(pruned.tr)
```

```
##
## Classification tree:
## rpart(formula = Competitive ~ ., data = train_data, method = "class")
##
## Variables actually used in tree construction:
## [1] Category    ClosePrice  currency    endDay      OpenPrice
## [6] sellerRating
##
## Root node error: 547/1183 = 0.46238
##
## n= 1183
##
##      CP nsplit rel error  xerror   xstd
## 1  0.297989      0  1.00000 1.00000 0.031350
## 2  0.100548      1  0.70201 0.70384 0.029461
## 3  0.063071      2  0.60146 0.60329 0.028200
## 4  0.053016      4  0.47532 0.52102 0.026889
## 5  0.031079      5  0.42230 0.48995 0.026321
## 6  0.020110      7  0.36015 0.43327 0.025167
## 7  0.017367      8  0.34004 0.40585 0.024550
## 8  0.014625     10  0.30530 0.35649 0.023330
## 9  0.013711     11  0.29068 0.31627 0.022218
## 10 0.010000     13  0.26325 0.29982 0.021728
```

```
prp(pruned.tr)
```



```
# Pruning the tree according to the optimal cp from part (c)
if (identical(tr, pruned.tr)) {
  # If the pruned tree is identical to the full tree, no changes occurred
  cat("The full tree was already optimal; pruning did not lead to any changes.")
} else {
  # Show the fit result after pruning
  printcp(pruned.tr)

  # Show pruned classification tree
  prp(pruned.tr)
}
```

```
## The full tree was already optimal; pruning did not lead to any changes.
```

The full tree was already optimal; pruning did not lead to any changes. It suggests that the initial tree has a balance of complexity that is sufficient to model the data without overfitting.

f. Calculate the complexity parameter (cp) with 1-standard error rule

```
#min_error_cp <- tr$cptable[which.min(tr$cptable[, "xerror"]), "CP"]
#se_rule_cp <- min_error_cp + tr$cptable[which.min(tr$cptable[, "xerror"]), "xstd"]
cp_error_rule=0.34601+0.023593
cat("The complexity parameter (cp) with 1-standard error rule:",cp_error_rule)
```

```
## The complexity parameter (cp) with 1-standard error rule: 0.369603
```

```
# Prune the tree according to the new optimal cp using 1-standard error rule
pruned_tr_se <- prune(tr, cp = cp_error_rule)
# Show the fit result after pruning using 1-standard error rule
printcp(pruned_tr_se)
```

```
##
## Classification tree:
## rpart(formula = Competitive ~ ., data = train_data, method = "class")
##
## Variables actually used in tree construction:
## character(0)
##
## Root node error: 547/1183 = 0.46238
##
## n= 1183
##
##      CP nsplit rel error xerror    xstd
## 1 0.3696      0      1      1 0.03135
```

```
# Compare with the original optimal cp
if (cp_error_rule < lowest_cp) {
  cat("Using 1-standard error rule, the optimal complexity parameter (cp) has changed to:", se_rule_cp)
} else if (cp_error_rule == lowest_cp) {
  cat("Using 1-standard error rule, the optimal complexity parameter (cp) remains the same.")
} else {
  cat("Using 1-standard error rule, the optimal complexity parameter (cp) remains the same as before.")
}
```

```
## Using 1-standard error rule, the optimal complexity parameter (cp) remains the same as before.
```

I decided to use the pruned tree for my model since both the trees are same

g. Determine the number of splits after pruning the tree and the training error rate The number of splits is 14 as we can see from the decision tree above and the training error rate is 0.29982 from the table.

h. Predict the response on the test data

```
# Predict response on the test data
test_pred <- predict(pruned.tr, newdata = test_data, type = "class")
# Produce confusion matrix
confusion_matrix <- table(test_pred, test_data$Competitive)
confusion_matrix
```

```
##
## test_pred  0   1
##      0 321  78
##      1  38 352
```

```
# Calculate test error rate (misclassification rate)
test_error_rate <- 1 - sum(diag(confusion_matrix)) / sum(confusion_matrix)
cat("Test Error Rate:", test_error_rate)
```

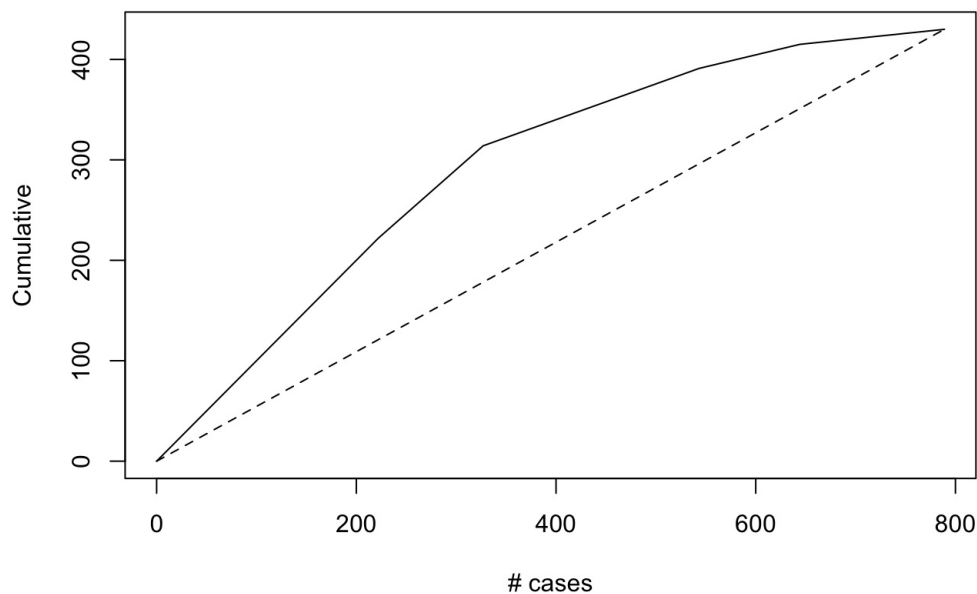
```
## Test Error Rate: 0.1470215
```

i. Produce a lift chart for the test data

```
# Produce predicted probabilities by specifying type="prob"
pred.prune.ctr.prob <- predict(pruned.tr, test_data, type = "prob")
# Output the predicted probabilities of class label "yes"
mod.pred <- pred.prune.ctr.prob[, 2]
# Create a gains table
gain <- gains(test_data$Competitive, mod.pred, groups = 5)
gain
```

##	Depth								
##	of		Cume	Mean	Cume	Cume Pct	Lift	Cume	Mean
##	File	N	N	Resp	Mean	of Total	Index	Lift	Model
##					Resp	Resp			Score
##	28	222	222	1.00	1.00	51.6%	183	183	0.99
##	41	105	327	0.88	0.96	73.0%	161	176	0.96
##	69	216	543	0.36	0.72	90.9%	65	132	0.38
##	82	101	644	0.24	0.64	96.5%	44	118	0.19
##	100	145	789	0.10	0.54	100.0%	19	100	0.12

```
# Create a lift chart
plot(c(0, gain$cume.pct.of.total * sum(test_data$Competitive)) ~ c(0, gain$cume.obs),
     xlab = "# cases", ylab = "Cumulative", main = "", type = "l")
lines(c(0, sum(test_data$Competitive)) ~ c(0, dim(test_data)[1]), lty = 2)
```



Since the lift chart of the tree model is higher than that of the baseline model, it is evident that the tree model's predictive performance is in correctly classifying competitive auctions.

Question 2

```
# Load necessary libraries
library(MASS)
library(tree)
library(randomForest)
```

```
## randomForest 4.7-1.1
```

```
## Type rfNews() to see new features/changes/bug fixes.
```

```
##
## Attaching package: 'randomForest'
```

```
## The following object is masked from 'package:rattle':
##
## importance
```

```
## The following object is masked from 'package:ggplot2':
##
## margin
```

```
# Load the Boston dataset
data(Boston)
```

- Randomly split the data into train (60%) and test (40%) set

```
set.seed(123)
#train_index <- sample(1:nrow(Boston), 0.6 * nrow(Boston))
train_index <- sample(c(1:dim(Boston)[1]), dim(Boston)[1]*0.6)
bos.train_data <- Boston[train_index, ]
bos.test_data <- Boston[-train_index, ]
```

b. Build a CART model on training observations

```
bosCART <- rpart(medv ~ ., data = bos.train_data, method="class")
# Predict on validation
test_pred <- predict(bosCART, newdata = bos.test_data)

# Calculate MSE
MSE <- mean((test_pred - bos.test_data$medv)^2)
MSE
```

```
## [1] 598.0604
```

c. Build a Bag-CART model using training observations

```
set.seed(123)
# Bag-CART model using training obs
bag.mod <- randomForest(medv ~ ., data = Boston, subset = train_index, mtry = 13, importance = T)
bag.mod
```

```
##
## Call:
## randomForest(formula = medv ~ ., data = Boston, mtry = 13, importance = T,      subset = train_index)
##              Type of random forest: regression
##              Number of trees: 500
## No. of variables tried at each split: 13
##
##              Mean of squared residuals: 11.61691
##              % Var explained: 85.94
```

```
# Predict on validation
valid_pred_bag <- predict(bag.mod, newdata = bos.test_data)

# Calculate MSE
MSE.bag <- mean((valid_pred_bag - bos.test_data$medv)^2)
MSE.bag
```

```
## [1] 17.88279
```

```
#Important variables
importance(bag.mod)
```

```
##              %IncMSE IncNodePurity
## crim      12.9974819      817.88830
## zn         4.7063703       40.32563
## indus      8.3045381      124.79258
## chas       0.6405165       26.86188
## nox        12.4933755      237.48950
## rm        49.2140860     11886.87120
## age       12.0052098      299.79830
## dis       18.6632015     1316.99772
## rad        1.9955304       74.43340
## tax       13.9310778      352.03514
## ptratio   12.9221878      196.91760
## black      3.0685164       214.65062
## lstat     35.3761414     9196.71440
```

```
varImpPlot(bag.mod)
```

bag.mod

